TECHNICAL PROJECT REPORT

ISM AttendanceCheck

Attendance & Payroll Management System

DATE	VERSION	REPOSITORY
April 21, 2026	1.0.0	ISM_AttendanceCheck

1 Project Overview

ISM AttendanceCheck is a full-stack, enterprise-grade web application designed to manage employee attendance, leave requests, and payroll calculations for Vietnamese businesses. It enforces **Vietnam statutory compliance** (BHXH, BHYT, BHTN, PIT), supports Microsoft Azure SSO login, and operates entirely in the **GMT+7 timezone**.

Repository: https://github.com/TheeVinhh/ISM_AttendanceCheck | **Branch:** main | **Files committed:** 62 source files

Attendance Tracking

Daily check-in/check-out with late detection, configurable shifts, and admin overrides.

Leave Management

Paid & unpaid leave requests, half-day support, balance validation, approval workflow.

Payroll Calculation

Vietnam-compliant payroll with OT rates, PIT brackets, and mandatory insurance deductions.

Role-Based Access

JWT + RBAC middleware ensuring employees only access their own data.

Azure SSO

Microsoft MSAL integration for enterprise single-sign-on alongside local auth.

Reports & Export

Monthly attendance reports and payroll CSV export for accounting.

2 System Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                        CLIENT  (Browser)                     │
│        React 18 + TypeScript + Vite   |   Tailwind CSS       │
│        Axios HTTP Client              |   MSAL (Azure SSO)   │
│                                                              │
│  Pages: Login | Dashboard | Calendar | Leaves | Payroll      │
│        AdminDashboard | Employees | AdminLeaves | Reports |   |
│        WorkingHours | AdminPayroll                           │
└──────────────────────────┬───────────────────────────────────┘
                           │  HTTPS  REST API  (JWT in headers)
┌──────────────────────────▼───────────────────────────────────┐
│                    SERVER  (Node.js + Express + TypeScript)  │
│                                                              │
│   ┌──────────────────────────────────────────────────────┐   |
│   │     Auth Middleware (JWT verify)  +  RBAC Middleware   |   │
│   └──────────────────────────────────────────────────────┘   |
│                                                              │
│  /api/auth  │  /api/attendance  │  /api/leaves               │
│  /api/admin │  /api/payroll                                  │
│                                                              │
│  Controllers → Business Logic → Mongoose Models              │
└──────────────────────────┬───────────────────────────────────┘
                           │  Mongoose ODM
┌──────────────────────────▼───────────────────────────────────┐
│                       DATABASE  (MongoDB)                    │
│                                                              │
│  Collections: Users | Attendance | LeaveRequests | PayrollRuns │
│               Departments | OvertimePolicies | WorkingHours  │
│               EmployeePayProfiles                            │
└──────────────────────────────────────────────────────────────┘
```

Technology Stack

Layer	Technology	Purpose
Frontend	React 18 + TypeScript + Vite	UI, routing, state management
Styling	Tailwind CSS + PostCSS	Utility-first responsive design
HTTP Client	Axios	API calls with interceptors
Auth (SSO)	MSAL (Microsoft Authentication Library)	Azure Active Directory login
Backend	Node.js + Express + TypeScript	REST API server
Database	MongoDB + Mongoose ODM	Document storage & schemas
Auth (local)	JWT + Bcrypt	Token auth & password hashing
Build	Vite (frontend) + tsc (backend)	Compilation & bundling

3 Project Structure

```
attendance-system/
├── backend/
│   ├── src/
│   │   ├── controllers/        # Business logic per feature
│   │   │   ├── auth.controller.ts
│   │   │   ├── attendance.controller.ts
│   │   │   ├── leave.controller.ts
│   │   │   ├── admin.controller.ts
│   │   │   └── payroll.controller.ts
│   │   ├── middleware/
│   │   │   ├── auth.middleware.ts   # JWT verification
│   │   │   └── rbac.middleware.ts   # Role enforcement
│   │   ├── models/             # MongoDB Mongoose schemas
│   │   ├── routes/             # Express route definitions
│   │   ├── types/              # Shared TypeScript types
│   │   └── utils/timezone.ts   # GMT+7 timezone helpers
│   └── scripts/create-admin.ts # One-time admin seeding
└── frontend/
    └── src/
        ├── api/axios.ts        # Configured Axios instance
        ├── components/         # Layout, Sidebar, Login button
        ├── config/msalConfig.ts
        ├── contexts/           # AuthContext, ThemeContext
        ├── pages/              # All 11 page components
        └── types/              # Frontend TypeScript types
```

4 User Roles & Permissions

The system implements two roles enforced by RBAC middleware on every protected route.

👤 Employee

- ✓ Register and log in (local or Azure SSO)
- ✓ View own profile (`/api/auth/me`)
- ✓ Check in and check out daily
- ✓ View today's attendance status
- ✓ View own monthly attendance history
- ✓ Submit paid & unpaid leave requests
- ✓ View own leave requests and balances
- ✓ View own payslip (approved runs only)
- ✗ View other employees' data
- ✗ Approve or reject leaves
- ✗ Create or manage payroll runs
- ✗ Access admin dashboard or reports

🛡️ Admin

- ✓ All employee permissions above
- ✓ View company-wide dashboard metrics
- ✓ View all employees + today's status
- ✓ View & generate monthly attendance reports
- ✓ Reset or manually update any attendance record
- ✓ View all leave requests company-wide
- ✓ Approve or reject leave requests
- ✓ Configure working hours & shift schedule
- ✓ Manage departments & overtime policies
- ✓ Create & version employee pay profiles
- ✓ Create, adjust, approve & export payroll runs
- ✓ Simulate payroll before committing a run

Leave Entitlements (Employee)

Leave Type	Annual Cap	Half-Day	Balance Validated
Paid leave (`with_salary`)	12 days/year	Yes	Yes – request rejected if exceeded
Unpaid leave (`without_salary`)	31 days/year	Yes	Yes – request rejected if exceeded

5 API Routes

Authentication – `/api/auth`

Method	Endpoint	Access	Description
POST	/register	Public	Register a new employee account (local auth)
POST	/login	Public	Login with email and password, returns JWT
POST	/azure	Public	Microsoft SSO login via MSAL token exchange
GET	/me	Authenticated	Get current user's profile

Attendance – `/api/attendance`

Method	Endpoint	Access	Description
POST	/checkin	Employee	Record check-in time for today
POST	/checkout	Employee	Record check-out time for today
GET	/today	Employee	Get current user's attendance for today
GET	/monthly?year=&month=	Employee	Get current user's monthly attendance history
GET	/all	Admin only	Get all employees' attendance for today

Leaves – `/api/leaves`

Method	Endpoint	Access	Description
POST	/	Employee	Submit a new leave request (validates balance)
GET	/my	Employee	Get current user's leave requests
GET	/	Admin only	Get all leave requests company-wide
PATCH	/:id	Admin only	Approve or reject a leave request

Admin – `/api/admin`

Method	Endpoint	Description
GET	/metrics	Dashboard KPIs: present count, pending leaves, payroll status
GET	/employees	List all employees with today's attendance status
GET	/attendance-report?year=&month=	Full monthly attendance report across all employees
GET	/employee/:userId/calendar	View a specific employee's attendance calendar
GET	/working-hours	Get current working hours configuration
PUT	/working-hours	Update shift schedule and late threshold
POST	/reset-attendance/:userId/:date	Reset attendance for a specific employee on a date
PUT	/attendance/:attendanceId/status	Manually set attendance status (present/late/absent/half-day)

Payroll – `/api/payroll`

Method	Endpoint	Access	Description
GET	/departments	Admin	List all departments
POST	/departments	Admin	Create department
PUT	/departments/:id	Admin	Update department
DELETE	/departments/:id	Admin	Delete department
GET	/ot-policies	Admin	List overtime policies
POST	/ot-policies	Admin	Create OT policy
PUT	/ot-policies/:id	Admin	Update OT policy
DELETE	/ot-policies/:id	Admin	Delete OT policy
GET	/profiles	Admin	List all employee pay profiles
GET	/profiles/:userId/history	Admin / Own	Get version history of a pay profile
POST	/profiles/:userId	Admin	Create new versioned pay profile for employee
GET	/runs	Role-filtered	List payroll runs (admins see all)
POST	/runs	Admin	Create a new payroll run (draft status)
GET	/runs/:runId	Role-filtered	View payroll run details
POST	/runs/:runId/submit	Admin	Submit run for review
POST	/runs/:runId/approve	Admin	Approve and lock the payroll run
PUT	/runs/:runId/entries/:userId	Admin	Adjust entry (draft/under_review only)
GET	/runs/:runId/export-csv	Admin	Export approved payroll to CSV
GET	/runs/:runId/my-payslip	Employee	View own payslip for a run
GET	/simulate	Admin	Simulate payroll without creating a run

6 Data Models

User

Field	Type	Notes
fullName	String	Required
email	String	Unique, indexed
role	enum	`employee` | `admin`
authProvider	enum	`local` | `azure`
passwordHash	String	Null for Azure users
createdAt / updatedAt	Date	Auto-managed timestamps

Attendance

Field	Type	Notes
userId	ObjectId → User	Required
date	String (YYYY-MM-DD)	GMT+7 date
checkInTime / checkOutTime	Date	UTC stored, GMT+7 displayed
status	enum	`present` | `absent` | `late` | `half-day`
notes	String	Optional admin note
Unique index	—	userId + date (one record per day)

LeaveRequest

Field	Type	Notes
userId	ObjectId → User	Required
date	String (YYYY-MM-DD)	Leave date
leaveType	enum	`with_salary` | `without_salary`
period	enum	`full_day` | `half_day_morning` | `half_day_afternoon`
reason	String	Employee-provided
status	enum	`pending` | `approved` | `rejected`
reviewedBy	ObjectId → User	Admin who acted on request

EmployeePayProfile (Versioned)

Field	Type	Notes
userId	ObjectId → User	Employee reference
payType	enum	`hourly` | `salaried` | `contract`
department / jobTitle / grade	String	Org structure fields
baseHourlyRate / monthlySalary / contractAmount	Number	One used depending on payType
allowances[] / deductions[]	Array	Flexible with taxable flag, categories
numberOfDependents	Number	For PIT dependent relief
overtimePolicyId	ObjectId → OvertimePolicy	Linked OT rules
effectiveFrom / effectiveTo	Date	Version date range
status	enum	`draft` | `active` | `superseded`
currency	String	Default: VND

PayrollRun

Field	Type	Notes
year / month	Number	Period identifier
status	enum	`draft` → `under_review` → `approved` → `exported`
entries[]	Array (embedded)	Per-employee payroll entries
totalGrossPay / totalNetPay	Number	Run-level totals
createdBy / approvedBy	ObjectId → User	Audit trail
approvedAt / exportedAt	Date	Timestamp audit

7 Payroll Engine (Vietnam Compliance)

Each payroll entry calculates the following in sequence:

Step 1 – Work Summary

- Counts **present**, **late**, **absent**, and **leave** days from attendance records
- **Early-arrival clamping:** hours before scheduled shift start are excluded from paid time
- Categorises hours as regular, overtime (weekday), or weekend

Step 2 – Base Pay

- **Salaried:** monthly salary prorated by working days attended
- **Hourly:** rate × regular hours worked
- **Contract:** fixed contract amount (not prorated)

Step 3 – Overtime Pay

- Daily OT threshold (e.g., 8 hrs) → excess billed at daily OT rate (e.g., 150%)
- Weekly OT threshold → additional premium applies
- Weekend hours billed at weekend rate (e.g., 200%)
- Holiday hours billed at holiday rate
- Monthly OT cap enforced per policy

Step 4 – Mandatory Insurance Deductions

Insurance	Employee Rate	Employer Rate	Ceiling
Social Insurance (BHXH)	8%	17.5%	46,800,000 VND (20× base salary)
Health Insurance (BHYT)	1.5%	3%	
Unemployment Insurance (BHTN)	1%	1%	

Step 5 – Personal Income Tax (PIT)

Item	Amount (VND/month)
Personal relief	11,000,000
Dependent relief (per dependent)	4,400,000
PIT brackets	Progressive 5% → 35%

Payroll Workflow

```
[Admin creates run]
        |
        ▼
    DRAFT  ──────► Entries can be viewed and adjusted
        |
        ▼
 UNDER_REVIEW ──► Entries can still be adjusted; awaiting final check
        |
        ▼
   APPROVED ──────► Entries are LOCKED; no further changes allowed
        |
        ▼
   EXPORTED ──────► CSV downloaded; audit record updated
```

8 Frontend Pages

Public

Route	Page	Description
/login	Login	Email/password form + Microsoft SSO button

Employee Pages (Authenticated)

Route	Page	Description
/dashboard	Dashboard	Check-in/check-out buttons, today's status, attendance cards
/calendar	CalendarPage	Monthly calendar view of personal attendance history
/leaves	LeavesPage	Submit leave requests, view leave balance, see request history with status
/payroll	PayrollPage	View approved payslips (gross pay, deductions, net pay breakdown)

Admin Pages (Authenticated + Admin Role)

Route	Page	Description
/admin/dashboard	AdminDashboard	Company KPIs: attendance rate, pending leaves, payroll status, absent count
/admin/employees	EmployeesPage	Full employee list with status badges; reset attendance action
/admin/leaves	AdminLeavesPage	All pending leave requests; approve/reject with one click
/admin/reports	ReportsPage	Monthly attendance reports across all staff; filter by year/month
/admin/working-hours	WorkingHoursPage	Configure shift times, two-shift mode, late threshold, break minutes
/admin/payroll	AdminPayroll (in PayrollPage)	Full payroll management: create runs, adjust entries, approve, export CSV

9 Security

Concern	Implementation
Authentication	JWT tokens verified on every protected request via auth middleware
Authorization	RBAC middleware rejects non-admin access to admin routes with HTTP 403
Password storage	Bcrypt hashing; plaintext passwords never stored or logged
Azure SSO users	No password hash stored; authenticated exclusively via MSAL token
Data isolation	Employee endpoints filter by authenticated userId from JWT payload
Payroll lock	Approved runs cannot be modified; API returns 409 Conflict on attempt
CORS	CORS configured on Express server